JENKINS HANDBOOK

Page 1 : Jenkins Fundamentals

★ What is Jenkins ?

Jenkins is an open-source automation server that helps to automate the parts of software development process.

It is widely used for Continuous Integration and Continuous Delivery (CI/CD).

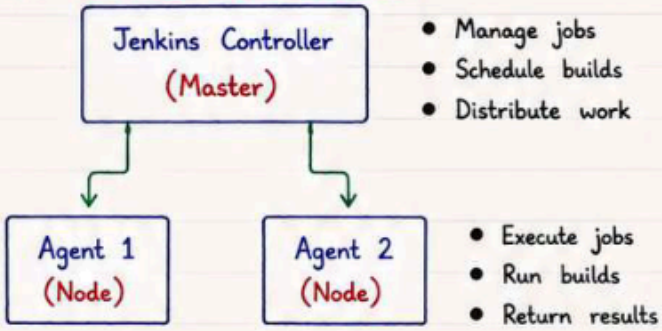
★ Why Jenkins ?

- ✓ Open Source & Free
- ✓ Easy to Install & Configure
- ✓ Extensible (Plugins)
- ✓ Supports CI/CD Pipeline
- ✓ Large Community Support
- ✓ Integrates with many tools

★ CI vs CD vs CI/CD

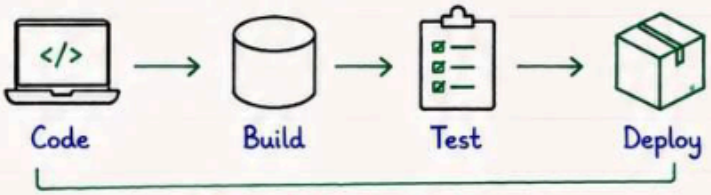
CI	CD	CI/CD
Continuous Integration	Continuous Delivery	Continuous Integration & Continuous Delivery
Developers integrate code frequently. Jenkins builds and tests automatically.	Every change that passes CI is auto delivered to a testing / staging environment.	Automate entire software delivery process from build to production.

★ Jenkins Architecture



- **Controller (Master)** : Controls the overall execution of jobs and manages agents.
- **Agent (Node)** : A machine that executes the jobs assigned by the controller.
- **Executor** : A worker process on a node that runs a build.

★ Basic Jenkins Workflow



Automated by Jenkins

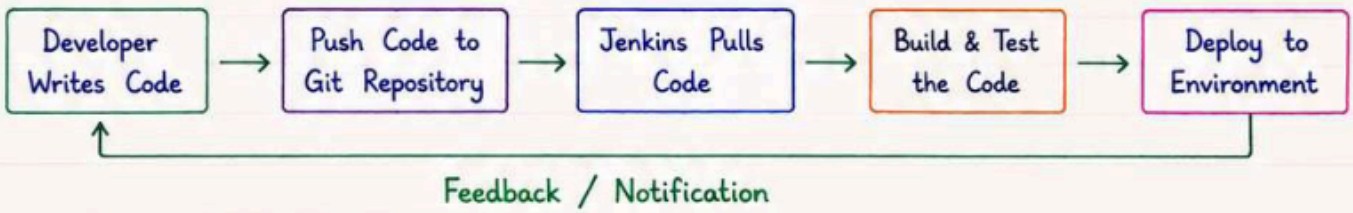
★ Job vs Build vs Pipeline

- **Job** : A task or project configured in Jenkins.
- **Build** : One execution of a job. Every run creates a new build.
- **Pipeline** : A suite of plugins that supports implementing and integrating continuous delivery pipelines.

★ Freestyle vs Pipeline

Freestyle Project	Pipeline Project
• GUI based configuration	• Code based (Jenkinsfile)
• Good for simple jobs	• Easy to maintain
• Hard to maintain complex workflows	• Supports complex CI/CD
• Not version controlled easily	• Version controlled in Git

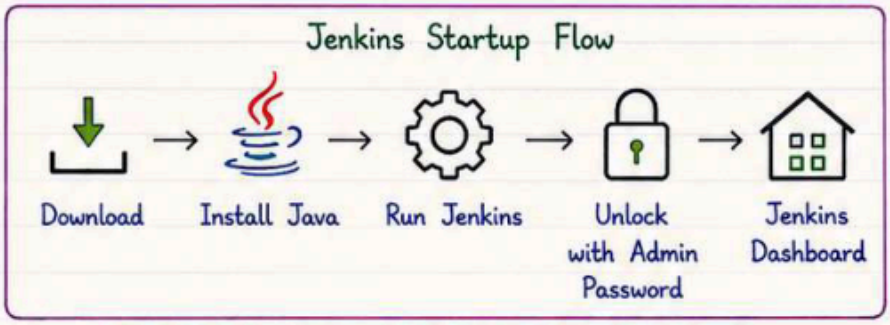
★ Jenkins Workflow (Real World)



Page 2 : Jenkins Setup & First Job

★ 1. Installing Jenkins

- Download Jenkins from official website : https://www.jenkins.io/download/
- Install Java (JDK 11 or 17+).
- Run Jenkins.
- Open in browser : http://localhost:8080
- Get initial admin password from console or file shown on screen.



Default URL : http://localhost:8080
Admin User : admin (after setup)

★ 2. Jenkins Dashboard Overview

New Item	→ Create new Jobs, Pipelines
People	→ View users
Build History	→ List of recent builds
Manage Jenkins	→ Global configuration, tools, settings
Credentials	→ Manage credentials
System Status	→ Check system health
Manage Plugins	→ Install / Manage plugins

Top Bar

- Search : Search jobs
- Bell Icon : Notifications
- Admin : User menu
- Log Out : Logout from Jenkins

★ 3. Create Your First Job (Freestyle Project)

Step 1 — Click on "New Item" on dashboard.

→ **Step 2** — Enter Job name (e.g., My-First-Job) Select "Freestyle project"

→ **Step 3** — Click OK. Now configure your job.

→ **Step 4** — Save the job.

★ 4. Configure Job

Section	What to Do	Example / Notes
General	• Add description if needed • Enable/Disable options	• This job builds my code. • Discard old builds, etc.
Source Code Management	• Select Git • Add Repository URL	Repository URL: https://github.com/username/repo.git
Build Triggers	• Build periodically • Poll SCM • GitHub hook trigger	• Schedule build (e.g., H/15 * * * *) • Check for SCM changes • Webhook from GitHub
Build Steps	• Add build steps • Example: Execute shell / Maven	mvn clean install or any build command
Post-build Actions	• Archive artifacts • Send email notification	Archive *.jar Send email on build status

★ 5. Run Your First Build

Click "Build Now"
↓
Jenkins executes the job
↓
Check Console Output for logs
↓
Build Status (Success / Failure)

Page 3 : Jenkins Pipeline Basics

★ 1. What is a Pipeline ?

- Pipeline is a suite of plugins that supports implementing and integrating continuous delivery pipelines.
- It allows you to define entire build process as code in a "Jenkinsfile".

★ 2. Pipeline Types

Declarative Pipeline	Scripted Pipeline
✓ Structured & easy to read	✓ More flexibility
✓ Uses predefined syntax	✓ Uses Groovy syntax
✓ Recommended for most users	✓ Advanced use cases

★ 3. What is Jenkinsfile ?

- Jenkinsfile is a text file that contains the definition of a Jenkins Pipeline.
- It is written in Groovy.
- It should be stored in the root of your source code repository.
- Jenkins reads this file and executes the pipeline.

Pipeline Flow



★ 4. Basic Declarative Pipeline Structure

```
pipeline {                          ⟶ Starts the pipeline
    agent any                       ⟷ Specifies where the pipeline will run (any available agent)
    stages {                        ⟷ Contains all the stages in the pipeline
        stage('Build') {            ⟷ A stage is a logical step in the pipeline
            steps {
                echo 'Building the code...'   ⟷ Contains the tasks/commands for that stage
            }                                 ⟶ Actual command/step (echo, sh, bat, etc.)
        }
        stage('Test') {
            steps {
                echo 'Running tests...'
            }
        }
    }
}
```

★ 5. Important Keywords in Declarative Pipeline

pipeline	agent	stages	stage	steps	post
• Top-level block that defines a pipeline.	• Defines where the pipeline will run.	• Contains one or more stages to be executed in order.	• A single logical step in the pipeline.	• Actual tasks or commands to be executed in that stage.	• Actions to run after pipeline completion (always, success, failure etc.)

★ 6. Example: Minimal Pipeline

```
pipeline {
    agent any
    stages {
        stage('Hello') {
            steps {
                echo 'Hello Jenkins Pipeline!'
            }
        }
    }
}
```

Explanation

→ Runs on any available agent
→ One stage named "Hello"
→ One step that prints message
→ Simple, valid pipeline

★ 7. Where can pipeline run ?



any → any available agent (Default)



node → specific node name (agent { label 'node1' })



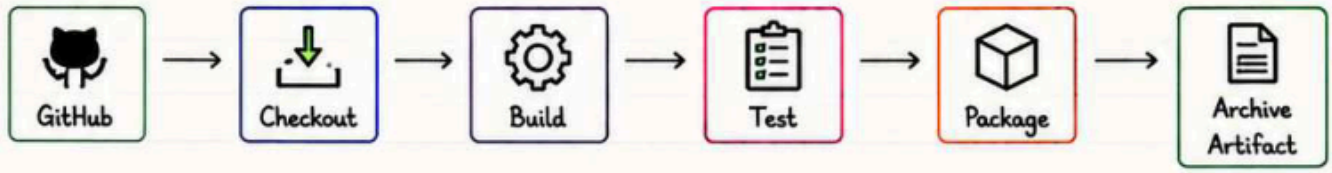
docker → inside a Docker container (agent { docker { ... } })



kubernetes → inside Kubernetes pod (agent { kubernetes { ... } })

Page 4 : Build a Real CI Pipeline (Maven Example)

★ 1. Real CI Pipeline Flow



★ 2. Sample Jenkinsfile (Declarative Pipeline)

```
pipeline {
    agent any
    environment {
        MAVEN_HOME = tool 'Maven3'
    }
    stages {
        stage('Checkout') {
            steps {
                git url: 'https://github.com/user/repo.git', branch: 'main'
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean compile'
            }
        }
        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }
        stage('Package') {
            steps {
                sh 'mvn package -DskipTests'
            }
        }
        stage('Archive Artifact') {
            steps {
                archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
            }
        }
    }
    post {
        success { echo 'Build Successful ☑' }
        failure { echo 'Build Failed ✗' }
        always { echo 'Build Completed' }
    }
}
```

★ 3. What happens in each stage?

Stage	What it does
Checkout	Pulls the code from Git repository.
Build	Compiles the source code.
Test	Runs unit tests (JUnit/TestNG).
Package	Creates the build artifact (JAR/WAR).
Archive Artifact	Saves the artifact for future use or download.

★ 4. Important Points

- Jenkinsfile should be in the root of the repository.
- Any change in code + push will trigger a new build (if configured).
- Artifacts are stored in Jenkins and can be used in later stages (e.g., deploy).
- Use `post {}` to handle success, failure, or cleanup tasks.
- Use `tools` section (Global Tool Configuration) to manage MAVEN, JDK, etc.

★ 5. Common Build Commands (Maven)

Command	Purpose
`mvn clean`	Cleans the target/ directory.
`mvn compile`	Compiles the source code.
`mvn test`	Runs the tests.
`mvn package`	Packages the application (JAR/WAR).
`mvn install`	Installs package to local repo.
`mvn deploy`	Deploys artifact to remote repo.

★ 6. Post Actions Example

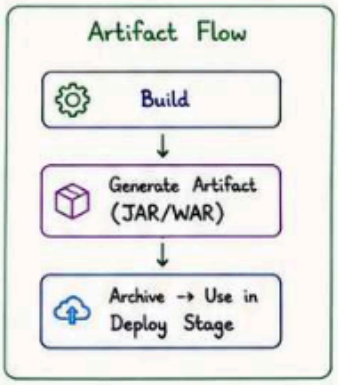
```
post {
    success {
        echo 'Build Successful ☑'
        // send mail, slack notification, etc.
    }
    failure {
        echo 'Build Failed ✗'
        // send instant alert
    }
    always {
        echo 'Build Completed ☑'
        // cleanup, archive logs
    }
}
```

★ 7. Artifact Example

- After build, artifact will be available in Jenkins → Workspace / Build → Archived Artifacts.
- Example artifact path:

  `target/my-app-1.0.0.jar`

- Can be used in next stage for Deployment.



Page 5 : Jenkins Triggers

★ 1. What are Build Triggers ?

- Triggers tell Jenkins WHEN to start a build.
- Without a trigger, job runs only manually.
- Triggers help automate builds based on events or schedule.

Trigger Flow (Example)



★ 2. Types of Build Triggers

Trigger Type	Description
Manual Trigger	User clicks "Build Now"... on Jenkins UI.
Build Periodically	Runs job at fixed intervals (using cron syntax).
Poll SCM	Jenkins checks SCM for changes at intervals.
GitHub Webhook	External event (push) triggers Jenkins using webhook.
Build When Upstream Project is Built	Triggers when an upstream job completes.
Build Other Projects	Triggers downstream jobs after this job completes.
Parameterized Trigger	Allows passing parameters while triggering the build.

★ 3. Configure GitHub Webhook (Example)

① Go to your GitHub Repository
② Settings → Webhooks → Add webhook
③ Payload URL : http://<jenkins-url>/github-webhook/
④ Content type : application/json
⑤ Select "Just the push event"
⑥ Click "Add webhook"



★ 4. Cron Syntax for "Build Periodically"

Format	Field
* * * * * 1 2 3 4 5	1 → Minute (0 - 59) 2 → Hour (0 - 23) 3 → Day of Month (1 - 31) 4 → Month (1 - 12) 5 → Day of Week (0 - 7) (0 or 7 = Sunday)

Examples	Meaning
H/5 * * * *	Every 5 minutes
H 2 * * *	Every day at 2 AM
H 2 * * 1-5	Every weekday at 2 AM
0 0 * * 0	Every Sunday at midnight
H * * * *	Every hour

Tip : Use H instead of a fixed number to spread load (e.g., H/5 instead of */5)

★ 5. Parameters in Jenkins

- Parameters make the pipeline dynamic.
- Values can be provided at build time.

Example in Declarative Pipeline

```
pipeline {
  agent any
  parameters {
    string(name: 'VERSION', defaultValue: '1.0', description: 'App Version')
    choice(name: 'ENV', choices: ['dev', 'qa', 'prod'], description: 'Environment')
    booleanParam(name: 'DEPLOY', defaultValue: true, description: 'Deploy App?')
  }
  stages {
    stage('Show Params') {
      steps {
        echo "Version: ${params.VERSION}"
        echo "Environment: ${params.ENV}"
        echo "Deploy: ${params.DEPLOY}"
      }
    }
  }
}
```

Parameter Types
- string
- choice
- booleanParam
- text
- password

★ Credentials in Jenkins

- Used to store sensitive information securely.
- Never store passwords or keys in Jenkinsfile.
- Access using withCredentials {}.

Credential Type	Use Cases	Example Usage in Pipeline
Username with password	Login to servers, tools, repositories.	withCredentials([usernamePassword(credentialsId: 'cred1', usernameVariable: 'USER', passwordVariable: 'PASS')]) { ... }
SSH Username with Private Key	Git over SSH, server access.	withCredentials([sshUserPrivateKey(credentialsId: 'ssh-key', keyFileVariable: 'KEY')]) { ... }
Secret Text	API Tokens, Secrets, Access Keys.	withCredentials([string(credentialsId: 'token', variable: 'TOKEN')]) { ... }
Secret File	Files like JSON, Key files, Certificates.	withCredentials([file(credentialsId: 'file-cred', variable: 'FILE')]) { ... }

Page 6 : Advanced Pipeline Concepts

1. Environment Variables

- Used to store key-value pairs.
- Can be global or stage level.

Example:

```
pipeline {
  agent any
  environment {
    APP_NAME = 'MyApp'
    BUILD_TYPE = 'Release'
  }
  stages {
    stage('Print') {
      steps {
        echo "App: ${APP_NAME}, Type: ${BUILD_TYPE}"
      }
    }
  }
}
```

Built-in Variables

- BUILD_NUMBER
- BUILD_URL
- WORKSPACE
- JOB_NAME
- NODE_NAME
- ... and many more

2. When Directive

- Controls when a stage should run.
- Evaluated before the stage starts.

Condition	Description
branch 'main'	Run only on main branch
branch 'feature/*'	Run on feature branches
environment name: 'prod'	Run when ENV = prod
expression { ... }	Custom boolean condition
changeset "**/*.java"	Run if .java files changed

Example:

```
stage('Deploy') {
  when {
    branch 'main'
  }
  steps {
    echo 'Deploying to production...'
  }
}
```

Tip

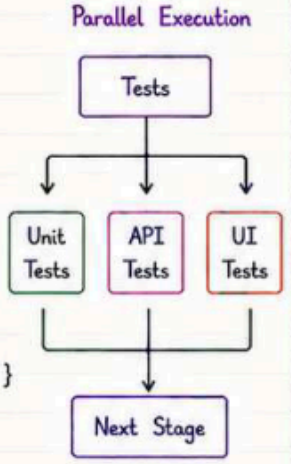
When is very useful to make pipeline smarter and avoid unnecessary executions.

3. Parallel Stages

- Run multiple stages at the same time.
- Saves build time.

Example:

```
stage('Tests') {
  parallel {
    stage('Unit Tests') {
      steps { sh 'mvn test' }
    }
    stage('API Tests') {
      steps { sh 'mvn verify' }
    }
    stage('UI Tests') {
      steps { sh 'mvn -f ui/pom.xml test' }
    }
  }
}
```



4. Input Step (Manual Approval)

- Pauses the pipeline and waits for user approval.
- Useful before production deployment.

```
stage('Approval') {
  steps {
    input message: 'Deploy to Production?',
          ok: 'Proceed',
          submitter: 'admin, devops-team'
  }
}
```



Note : submitter → users/groups allowed to approve.

5. Timeout & Retry

Timeout

- Abort stage if it takes too long.

```
stage('Long Running') {
  options {
    timeout(time: 30, unit: 'MINUTES')
  }
  steps {
    sh 'sh long-running-script.sh'
  }
}
```

Retry

- Retry a failed step/stage multiple times.

```
stage('Tests') {
  steps {
    retry(3) {
      sh 'mvn test'
    }
  }
}
```

6. Post Directive

- Defines actions to run after the pipeline or stage completes.
- Very useful for notifications, cleanup, archiving, etc.

Example:

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps { sh 'mvn clean package' }
    }
  }
  post {
    success { echo 'Build Successful ☑' }
    failure { echo 'Build Failed ✗' }
    always { echo 'Pipeline Completed' }
    unstable { echo 'Build Unstable ⚠' }
  }
}
```

Post Conditions	When it runs
success	When pipeline succeeds
failure	When pipeline fails
unstable	When build is unstable
always	Always (succeeds or fails)
changed	When build status changes

Typical Post Actions

- Send Email / Slack Notification
- Archive Artifacts / Logs
- Cleanup Workspace
- Publish Test Reports
- Upload to Server / S3

Key Takeaway

Advanced features make pipelines powerful, reliable and production ready.

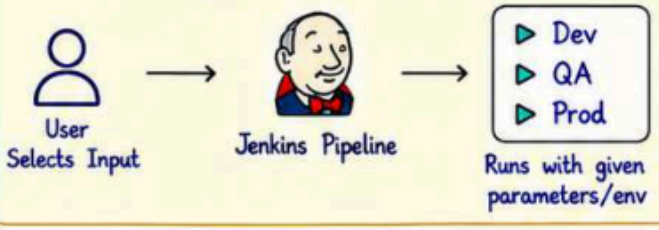
Use these features to write smart, reusable and efficient pipelines!

Page 7 : Jenkins Parameters, Environment & Credentials

★ 1. Why Parameters & Environment Variables ?

- Make pipeline flexible ☑
- Reuse same pipeline for multiple environments ☑
- Pass values at build time ☑
- Avoid hardcoding ☑

Dynamic Pipeline Flow



★ 2. Jenkins Parameters (Types)

Type	UI Element	Example / Use Case
String	abc	Version, Tag, Branch
Choice	▼	Environment (dev/qa/prod)
Boolean	☑	Run Tests (Yes/No)
Password	****	Hidden sensitive value
File	📄	Upload config file / script

Example : Parameters in Declarative Pipeline

```
pipeline {
  agent any
  parameters {
    choice(name: 'ENV', choices: ['dev', 'qa', 'prod'],
           description: 'Select Environment')
    string(name: 'VERSION', defaultValue: '1.0.0',
           description: 'App Version')
    booleanParam(name: 'RUN_TESTS', defaultValue: true,
                 description: 'Run Unit Tests')
  }
  stages {
    stage('Info') {
      steps {
        echo "ENV = ${params.ENV}"
        echo "Version = ${params.VERSION}"
        echo "Run Tests = ${params.RUN_TESTS}"
      }
    }
  }
}
```

★ 3. Environment Variables

- Store key-value pairs.
- Can be global or stage level.
- Useful for configuration and secrets (non-sensitive).

Example : Environment Variables

```
pipeline {
  agent any
  environment {
    APP_NAME = 'MyApp'
    BUILD_URL_VAR = "${env.BUILD_URL}"
  }
  stages {
    stage('Print') {
      steps {
        echo "App: ${env.APP_NAME}"
        echo "Build URL: ${env.BUILD_URL_VAR}"
      }
    }
  }
}
```

Useful Built-in Env Variables

- BUILD_NUMBER
- BUILD_URL
- JOB_NAME
- WORKSPACE
- NODE_NAME
- BRANCH_NAME
- ...

★ 4. Jenkins Credentials

- Used to store sensitive information securely.
- Never hardcode secrets in Jenkinsfile.
- Access using withCredentials {}.

Example : Using Credentials

```
pipeline {
  agent any
  stages {
    stage('Download from Private Repo') {
      steps {
        withCredentials([usernamePassword(
          credentialsId: 'git-cred',
          usernameVariable: 'GIT_USER',
          passwordVariable: 'GIT_PASS')]) {
          sh 'git clone https://$GIT_USER:$GIT_PASS@git.company.com/repo.git'
        }
      }
    }
  }
}
```

Credential Types

- ✓ Username / Password
- ✓ SSH Key
- ✓ Secret Text (API Token)
- ✓ Secret File
- ✓ Certificate

★ 5. Best Practices

- ✓ Keep Jenkinsfile in Git repository.
- ✓ Use parameters for dynamic values.
- ✓ Use credentials for all sensitive data.
- ✓ Avoid hardcoding passwords / tokens.
- ✓ Use environment variables for configuration.
- ✓ Provide proper defaults and descriptions.
- ✓ Keep pipeline reusable for dev, qa and prod.

★ 6. Quick Recap

- Parameters → User input at build time.
- Environment → Key-value config inside pipeline.
- Credentials → Secure way to use secrets.
- Together they make pipeline dynamic, secure and reusable !

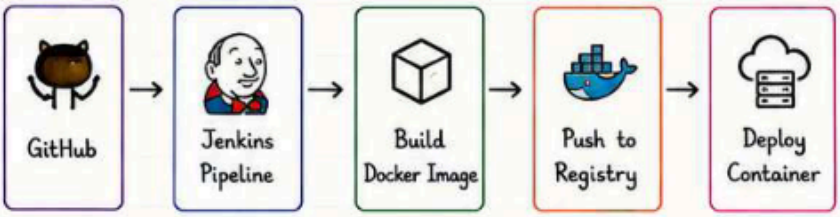
Page 8 : Jenkins + Docker + Real-World Integration

★ 1. Why Docker with Jenkins ?

- Consistent build environment
- Easy dependency management
- Lightweight & portable
- Same image in Dev, Test, Prod

Jenkins builds → Docker Image → Push to Registry → Deploy

2. High Level Flow



★ 3. Jenkins Pipeline with Docker (Example)

```
pipeline {
  agent any
  environment {
      DOCKER_IMAGE = "myapp"
      REGISTRY = "dockerhub-username"
      TAG = "${BUILD_NUMBER}"
  }
  stages {
     stage('Checkout') {
        steps {
           git url: 'https://github.com/user/repo.git', branch: 'main'
        }
     }
     stage('Build') {
        steps {
           sh 'mvn clean package -DskipTests'
        }
     }
     stage('Unit Test') {
        steps {
           sh 'mvn test'
        }
     }
     stage('Docker Build') {
        steps {
           sh "docker build -t ${REGISTRY}/${DOCKER_IMAGE}:${TAG} ."
        }
     }
     stage('Docker Push') {
        steps {
           withCredentials([usernamePassword(credentialsId: 'docker-creds',
             usernameVariable: 'USR', passwordVariable: 'PWD')]) {
              sh 'echo $PWD  docker login -u $USR --password-stdin'
              sh "docker push ${REGISTRY}/${DOCKER_IMAGE}:${TAG}"
           }
        }
     }
     stage('Deploy') {
        steps {
           sh "ssh user@server 'docker pull ${REGISTRY}/${DOCKER_IMAGE}:${TAG} && \
               docker stop myapp || true && \
               docker rm myapp || true && \
               docker run -d --name myapp -p 8080:8080 ${REGISTRY}/${DOCKER_IMAGE}:${TAG}'"
        }
     }
  }
  post {
     success { echo 'Pipeline completed successfully ☑' }
     failure { echo 'Pipeline failed ✗' }
  }
}
```

★ 4. Dockerfile Example

```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/myapp.jar myapp.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "myapp.jar"]
```

★ 5. What Happens in Each Step ?

Stage	What it Does
Checkout	Pulls code from GitHub repository.
Build	Builds the project using Maven.
Unit Test	Runs unit tests.
Docker Build	Creates a Docker image.
Docker Push	Pushes image to Docker Registry.
Deploy	Pulls image on server & runs container.

★ 6. Jenkins Credentials Used

- docker-creds → Docker Hub username & password
- ssh-server → SSH access to deployment server
- git-credentials → GitHub access (if private repo)

💡 Tip : Never hardcode secrets in Jenkinsfile !

★ 7. Example Folder Structure

```
myapp/
├── Jenkinsfile        ← Pipeline definition
├── Dockerfile         ← Docker build instructions
├── pom.xml            ← Maven configuration
├── src/               ← Source code
│   └── ...
└── target/            ← Build output (JAR file)
```

8. Docker Commands Used

- `docker build -t image:tag .` → Build image from Dockerfile
- `docker login -u USER --password-stdin` → Login to Docker Hub
- `docker push image:tag` → Push image to registry
- `docker pull image:tag` → Pull image from registry
- `docker run -d -p 8080:8080 image:tag` → Run container in detached mode

9. Benefits

- ✓ Same environment everywhere
- ✓ Easy rollbacks (use previous image)
- ✓ Scalable deployments
- ✓ Fast and reliable releases
- ✓ Ideal for microservices

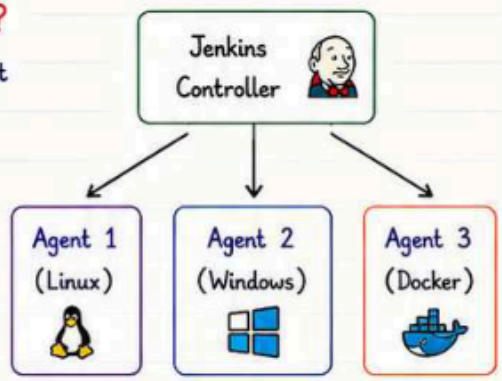


★ Key Takeaway : Jenkins + Docker = Automated, Consistent, Scalable & Reliable Delivery Pipeline

Page 9 : Jenkins Agents & Distributed Builds

1. What is an Agent ?

- Agent is a machine that runs the build.
- Offloads work from the controller.
- Improves performance & scalability.



2. Agent Types

Type	Description
Permanent Agent	Always connected to controller.
Dynamic Agent	Created on demand (Docker, K8s).
Inbound Agent	Agent connects to controller.
Outbound Agent	Controller connects to agent.

3. Example : Use Agent in Declarative Pipeline

```
pipeline {
    agent { label 'linux-agent' }
    environment {
        MAVEN_HOME = '/opt/maven'
    }
    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }
        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }
    }
}
```

Note : Label of agent must match the label configured in Jenkins.

4. How to Add a New Agent (Node)

1. Go to : Manage Jenkins → Manage Nodes and Clouds
2. Click on "New Node"
3. Enter Node name
4. Select "Permanent Agent"
5. Give the number of executors
6. Choose Remote root directory
7. Launch method → "Launch agents via SSH"
8. Provide Host, Credentials, Port
9. Save → Agent will be online

Tip : Use multiple agents to run builds in parallel → faster builds, better resource utilization !

5. Agent Labels in Jenkins

- Labels help Jenkins decide which agent to use.
- You can set multiple labels to an agent.

Example

Agent Name : linux-build-server
Labels : linux, maven, java17, docker

Use in Pipeline

```
agent {
    label 'linux && maven'
}
```

6. Docker Agent (Dynamic)

- Jenkins spins up a Docker container as an agent.
- Container is removed after build.

```
pipeline {
    agent {
        docker {
            image 'maven:3.9.6-eclipse-temurin-17'
            args '-v /var/run/docker.sock:/var/run/docker.sock'
        }
    }
    stages {
        stage('Build') { steps { sh 'mvn clean package' } }
    }
}
```

7. When to Use Agents ?

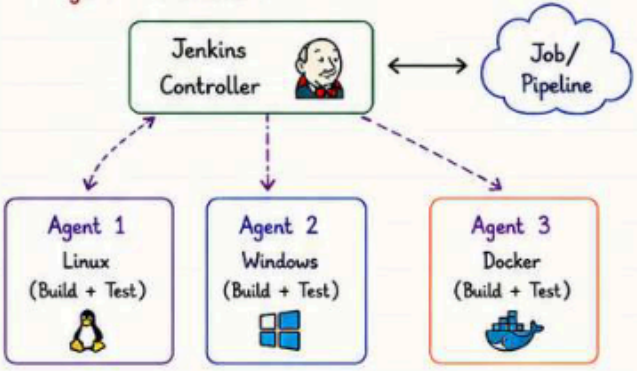
Use Case	Why Agent is Needed
Heavy Builds	Do not overload the controller.
Different OS	e.g. Windows build on Windows agent.
Docker Builds	Run Docker builds on Docker agent.
Parallel Builds	Run multiple jobs simultaneously.
Security	Keep controller secure & lightweight.

8. Best Practices

- ✓ Keep controller free from heavy builds.
- ✓ Use agents with proper labels.
- ✓ Use dynamic agents for scalability.
- ✓ Do not run Docker inside Docker on controller.
- ✓ Monitor agent usage & availability.
- ✓ Keep number of executors reasonable.

→ Better Performance & Scalability

9. Agent Architecture



Key Takeaway : Agents make Jenkins faster, scalable & production ready !

Page 10 : Jenkins – Best Practices & Summary

1. Best Practices

- ✓ Keep Jenkins & Plugins up to date.
- ✓ Use credentials & secrets securely.
- ✓ Use Pipeline as Code (Jenkinsfile in Git).
- ✓ Keep builds small, fast & independent.
- ✓ Use agents for parallel & distributed builds.
- ✓ Archive artifacts & logs for every build.
- ✓ Clean workspace after build.
- ✓ Monitor builds & setup notifications.
- ✓ Use quality gates & stages (Dev → Test → Prod).
- ✓ Backup Jenkins regularly.

2. Common Jenkins Plugins

Plugin	Purpose
Pipeline	Build Pipeline as Code
Git	Integrate with Git repositories
Docker	Build & run Docker images
Credentials Binding	Use credentials securely
Blue Ocean	Modern UI for Pipelines
Email Extension	Send email notifications
JUnit	Publish test results
SonarQube	Code quality analysis

3. Troubleshooting Tips

- Check Console Output for errors.
- Verify Jenkinsfile syntax.
- Check credentials & permissions.
- Ensure agent is online.
- Check disk space & memory.
- Restart Jenkins if needed.

Tip : Most issues are due to permissions, credentials or wrong configuration.

4. Useful Jenkins Environment Variables

Variable	Description
BUILD_NUMBER	Current build number
BUILD_ID	Unique ID of the build
JOB_NAME	Name of the job
WORKSPACE	Workspace directory
BUILD_URL	URL of the current build
NODE_NAME	Name of the node/agent

5. Typical Jenkins Pipeline Flow (End to End)

1. **Commit** — Code push to Git
2. **Trigger** — Webhook / Poll / Manual Trigger
3. **Build** — Compile / Build the code
4. **Test** — Run Unit / API / Integration Tests
5. **Package** — Create artifact / Docker Image
6. **Deploy** — Deploy to Environment
7. **Notify** — Send Email / Slack Notification

6. Declarative vs Scripted Pipeline

Declarative Pipeline	Scripted Pipeline
Simpler syntax (pipeline {})	Groovy based (more flexible)
Easy to read & maintain	More control & complex logic
Built-in features (stages, steps)	Everything is code
Recommended for most use cases	Use for advanced scenarios

7. Security Best Practices

- Use Role-Based Access Control
- Store secrets in Credentials Store.
- Don't hardcode passwords / tokens.
- Use HTTPS for all connections.
- Regularly update Jenkins & plugins.
- Audit logs & monitor activities.

8. Summary

Jenkins helps automate the entire software delivery process from build to deploy.

Automate. Test. Deploy. Repeat.

Key Takeaway : Jenkins + Good Practices = Fast, Reliable & Automated Delivery !